UNIVERSIDAD DE CANTABRIA

INGENIERÍA DEL SOFTWARE II

Práctica 3

Alejandro López-Tormos Gervás

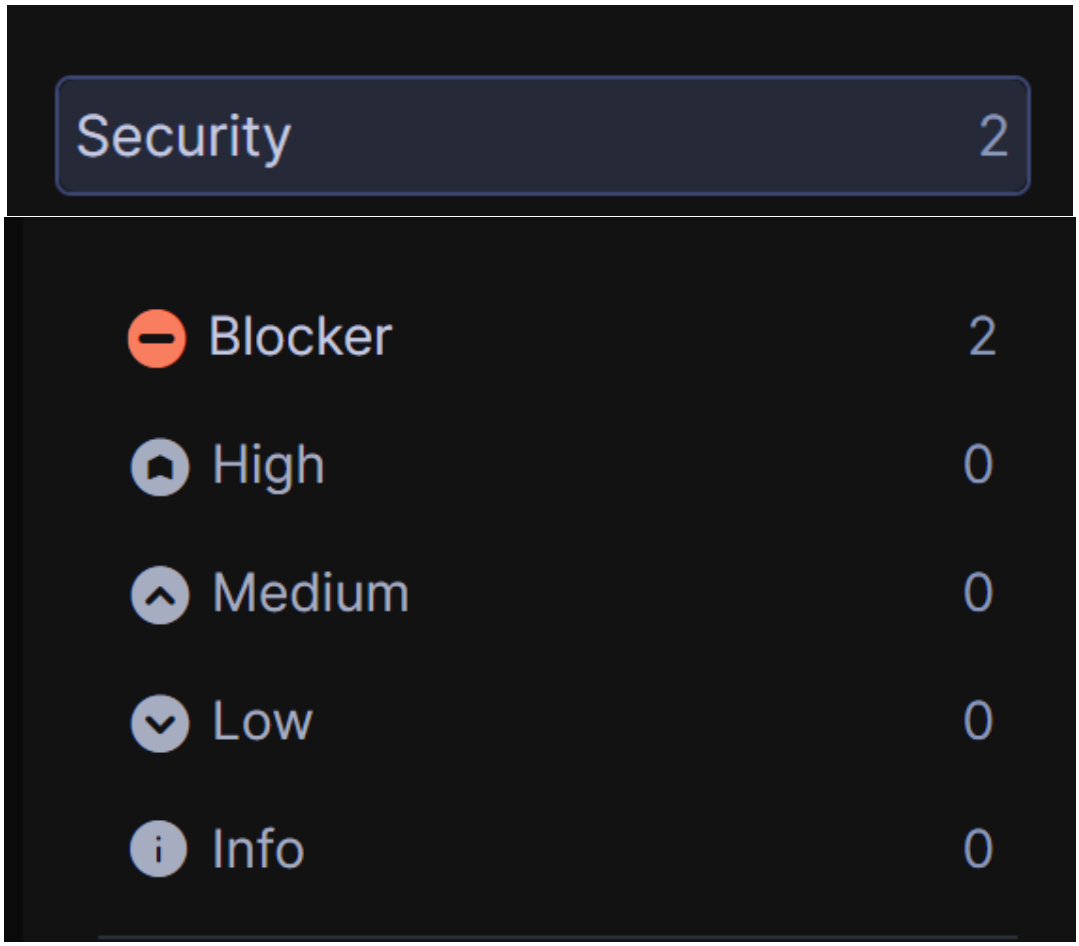
Date created:
Miércoles, 11 de marzo de 2026

1. Análisis de SonarQube

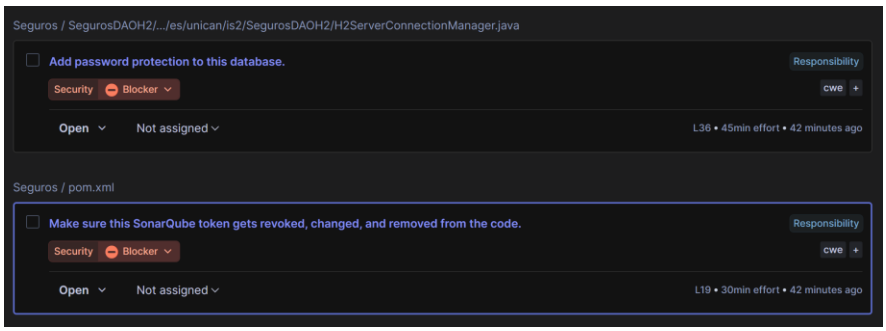
1.1. Análisis inicial de la aplicación

En SonarQube existen varios tipos de métricas cuando se habla de los tipos de error que califica SonarQube. Primeramente, según la gravedad de los problemas en el código estas pueden ser: blocker, high, medium, low o info. Seguidamente están las las que caracterizan al error según su naturaleza. “Software Quality” son las cuales indican los problemas de la calidad software, por ejemplo, sobre el acceso a personal no autorizado. “Reliability” son las que miran lo capaz que es el código de ser ejecutado y mantener el rendimiento del programa bajo diferentes condiciones. Por último, las “Maintainability” que indican la facilidad con la que puede reparar, mejorar y comprender el código del software.

En el apartado de “Security”, hay 2 errores los cuales son de tipo blocker.



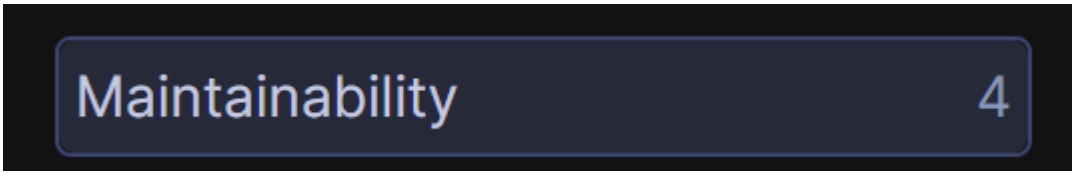
Estos errores son:

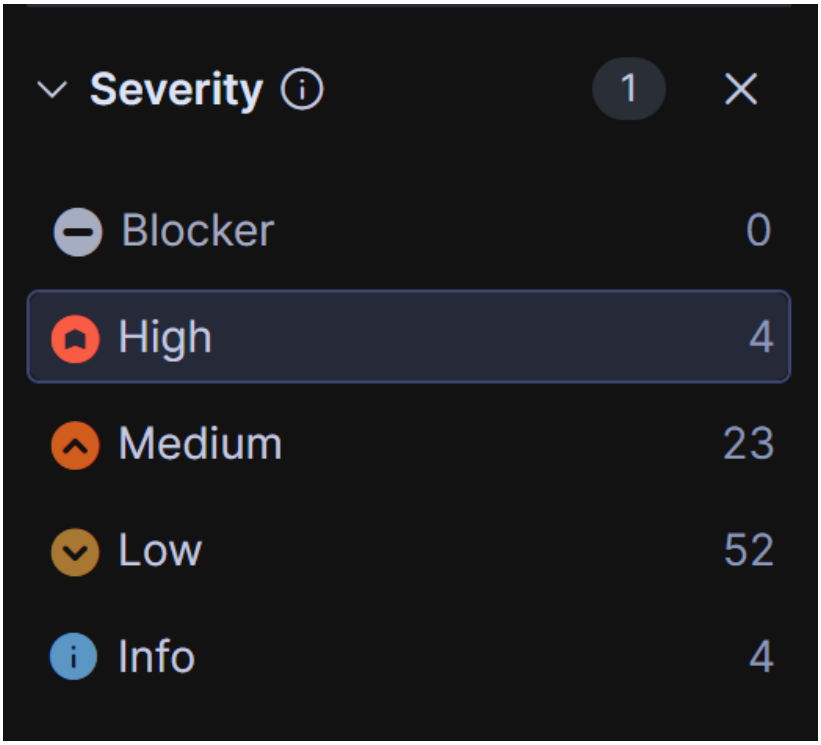


Se puede ver en que archivo se da el problema, el responsable que debería de resolver este problema y cuánto tiempo se tarda en resolverlo (estos tiempos pueden ser fatalistas). Por ejemplo, el primero se tardaría en resolver 45 minutos.

En el apartado de “Reliability” hay estos errores:

Estos errores son una escala menos que la de blocker. Todos estos errores se repiten siendo: “**Use try-with-resources or close this "Statement" in a "finally" clause.**” En diferentes partes del Código.



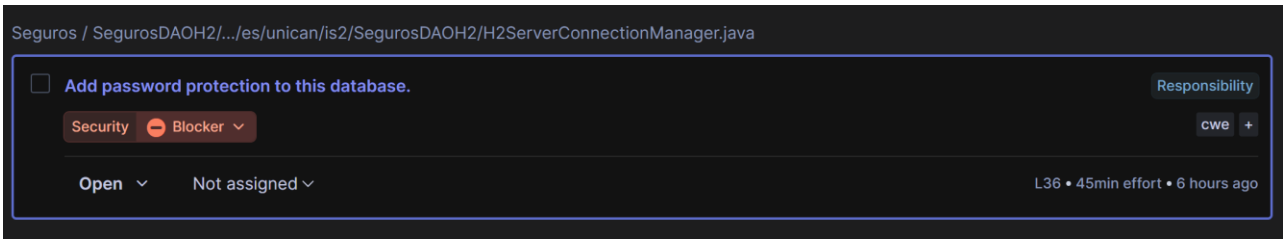


Aquí hay diferentes tipos de problemas. Primeramente, los “High” se refieren a problemas como hacer non-static o static algunas varibales en VistaAgente. Las “Medium” suelen ser eliminar asignaciones a variables inútiles, o algunas recomendaciones en las queries. Las “Low” indican cambiar el nombre de los paquetes, eliminar variables temporales y retornar el valor directamente o reordenar los modificadores para que cumplan la “Java Language Specification”. Por último, las de info recomiendan quitar modificadores públicos.

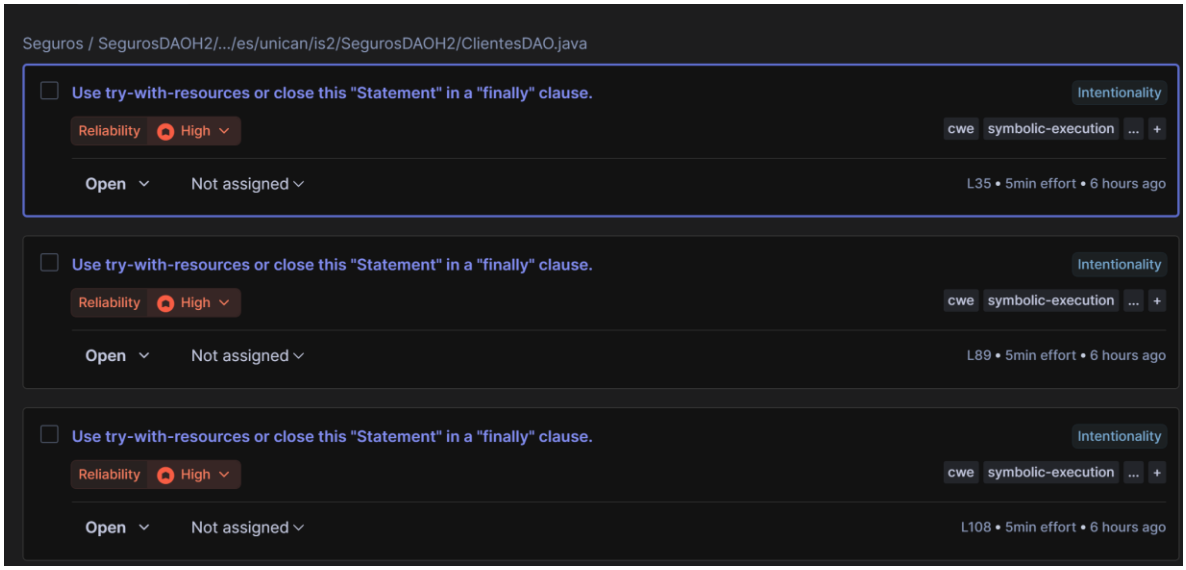
1.2. Plan de Mejora justificado y priorizado

Según se indica en el PDF proporcionado en la práctica, solo se disponen de 60 minutos para resolver los problemas. Como la duración para resolverlos todos es de aproximadamente de 1 día y 4 horas, se van a elegir los problemas más prioritarios. Los errores de “Blocker” son la mayor prioridad seguido de las “High”. El criterio para elegir las tareas será intentar ajustarlas lo máximo posible a los 60 minutos disponibles, haciendo cuantas más tareas posibles, es decir se priorizará el arreglo de tareas con menos tiempo dentro del mismo nivel.

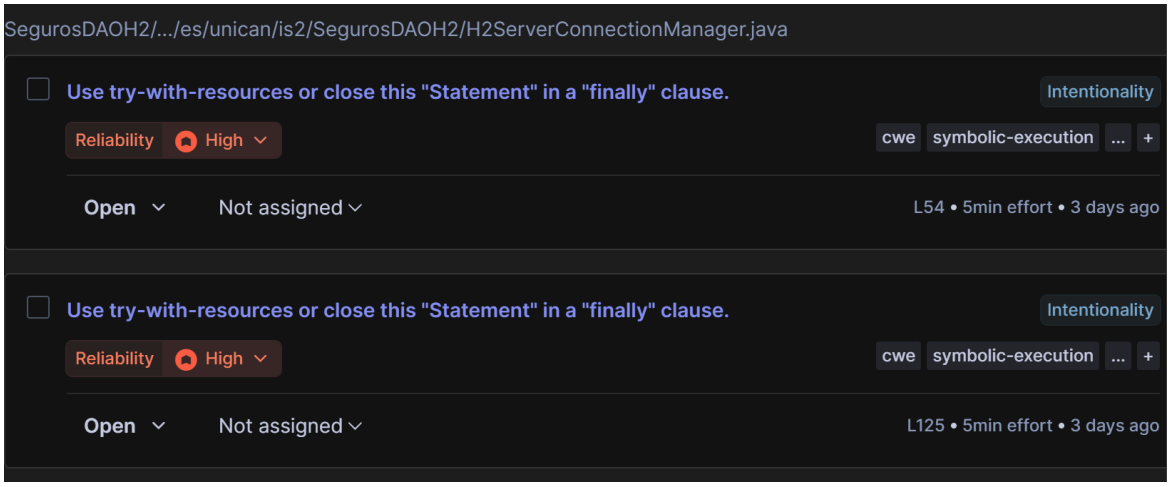
La primera tarea para solucionar sería “Add password protection to the database”. Afecta gravemente a la seguridad del proyecto y el nivel de severidad es máximo (“Blocker”). Sin embargo, como al ser una base de datos **H2 en memoria**, solo es accesible desde la propia aplicación y no hay conexiones externas posibles, por lo que la contraseña tiene un impacto de seguridad limitado en este caso concreto, así que en este caso la contraseña solo es simbólica y por lo tanto merece la pena arreglar otros fallos.



Las siguientes tareas serán de nivel “High”. En la clase “ClientesDAO” en el paquete “SegurosDAOH2”, hay tres errores similares “Use try-with-resources or close this “Statement” in a “finally” clause”, en total se tardará 15 minutos en total (5 minutos cada una). Para solucionar esto se utilizará el try-with-resources, porque queda mucho más legible en el código. El problema es que si ocurre una excepción antes de que se ejecute statement.close(), el Statement queda abierto consumiendo recursos innecesariamente. Declarar el Statement dentro del try, de forma que se cierra automáticamente al salir del bloque.



Las siguientes tareas son del mismo tipo, pero de la clase H2ServerConnectionManager.java. En total se tardarán 10 minutos en resolver estas incidencias.



Lo mismo pasa con SegurosDAO.java.

SegurosDAOH2/.../es/unican/is2/SegurosDAOH2/SegurosDAO.java

Use try-with-resources or close this "Statement" in a "finally" clause. Intentionality
Reliability High cwe symbolic-execution ... +
Open Not assigned L59 • 5min effort • 3 days ago

Use try-with-resources or close this "Statement" in a "finally" clause. Intentionality
Reliability High cwe symbolic-execution ... +
Open Not assigned L79 • 5min effort • 3 days ago

Use try-with-resources or close this "Statement" in a "finally" clause. Intentionality
Reliability High cwe symbolic-execution ... +
Open Not assigned L100 • 5min effort • 3 days ago

Dado que algunas tareas requieren media hora de esfuerzo, es preferible descartarlas y centrarse en las de menor duración. Es decir que de la clase VistaAgente.java solo se va a arreglar esta de 5 minutos y también es “high” y de tipo “Maintainability”. La solución de esta tarea ya es diferente de las previas expuestas. La propuesta es acceder a la constante estática a través de la clase en vez de con la instancia, además de hacer el import correspondiente.

Use static access with "javax.swing.WindowConstants" for "EXIT_ON_CLOSE". Intentionality
Maintainability High confusing +
Open Not assigned L53 • • 5min effort • 3 days ago

12 of 12 shown

Las tareas que siguen son de nivel “Medium”, se resolverán quitando la asignación inútil sobre la variable especificaciones. Las tareas están en SegurosDAO.java y afecta a la “Maintainability”

SegurosDAOH2/.../es/unican/is2/SegurosDAOH2/SegurosDAO.java

Remove this useless assignment to local variable "con". Intentionality

Maintainability Medium cwe cert ... +

Open Not assigned L33 • 1min effort • 3 days ago

Remove this useless assignment to local variable "old". Intentionality

Maintainability Medium cwe cert ... +

Open Not assigned L42 • 1min effort • 3 days ago

Remove this useless assignment to local variable "con". Intentionality

Maintainability Medium cwe cert ... +

Open Not assigned L44 • 1min effort • 3 days ago

Lo mismo, pero en la clase ClientesDAO.java

SegurosDAOH2/.../es/unican/is2/SegurosDAOH2/ClientesDAO.java

Remove this useless assignment to local variable "con". Intentionality

Maintainability Medium cwe cert ... +

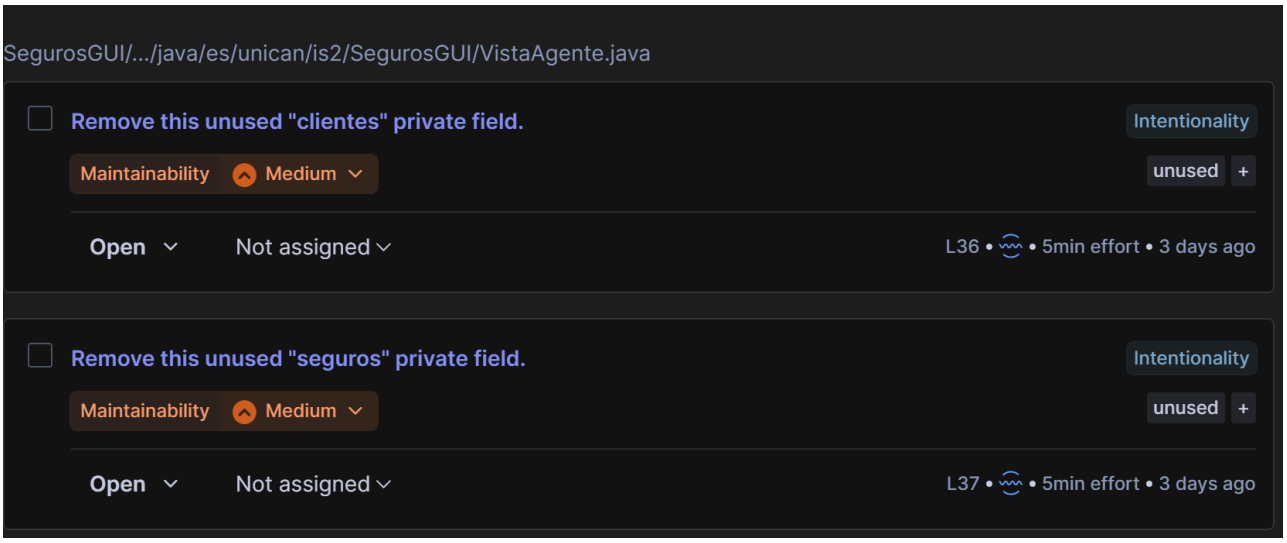
Open Not assigned L56 • 1min effort • 3 days ago

Remove this useless assignment to local variable "con". Intentionality

Maintainability Medium cwe cert ... +

Open Not assigned L78 • 1min effort • 3 days ago

Por último, en VistaAgente.java los dos errores “Medium” y de tipo “Maintainability” que se van a arreglar son los siguientes. Se tardará 10 minutos en total y la solución propuesta será quitar esas variables que no se utilizan en el código.



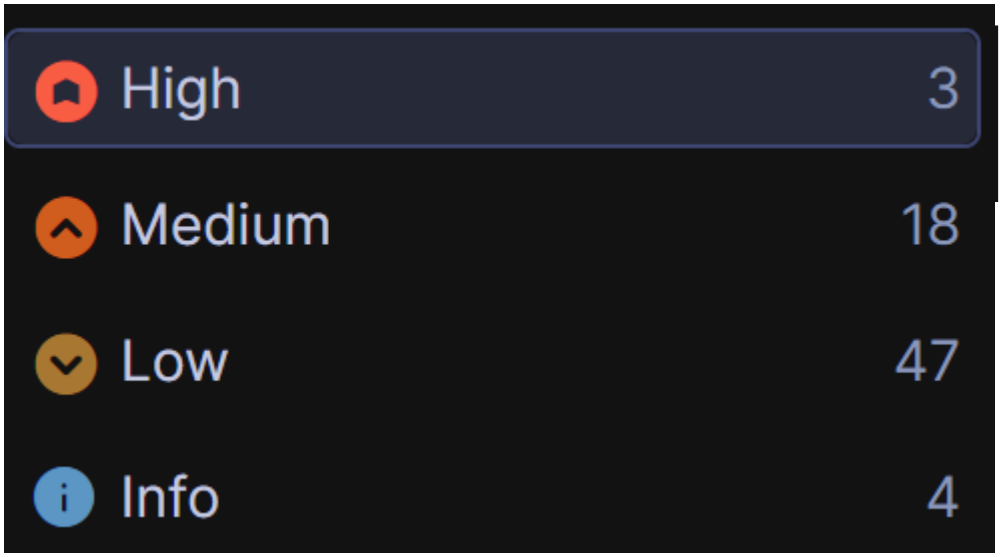
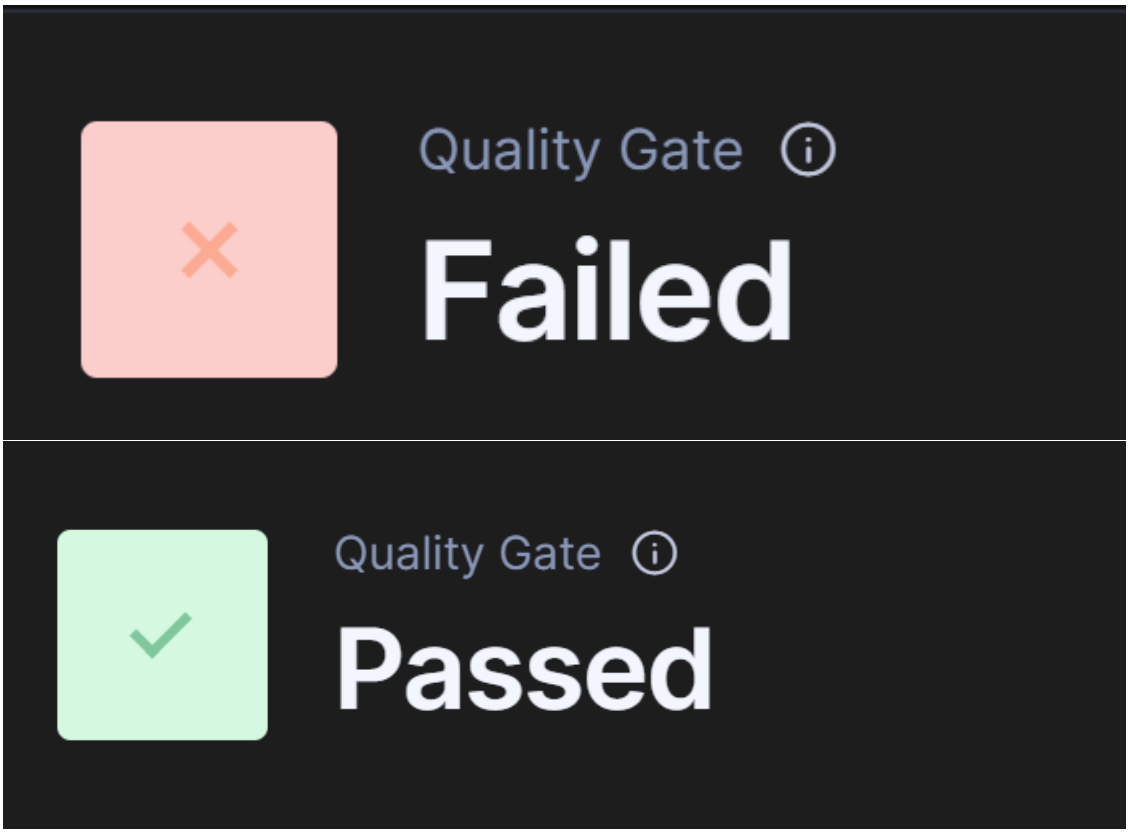
Las tareas para arreglar cumplen los 60 minutos que había permitidos así que no se podrá arreglar nada más. En resumen, en este análisis se van a resolver 9 incidencias de severidad “High” y 7 incidencias de nivel “Medium”

1.3. Registro de incidencias resueltas, descartadas y justificación

Se han resuelto las incidencias propuestas en el plan de mejora: 9 de severidad “High” y 5 de severidad “Medium”.
Cabe mencionar que SonarQube detecta adicionalmente 2 incidencias Medium en VistaAgente.java recomendando eliminar los campos privados clientes y seguros. Sin embargo, al aplicar esta corrección el proyecto deja de compilar, por lo que se tratan de falsos positivos y han sido descartadas.
(También he arreglado el error “blocker” para aprender a utilizar variables del sistema en java, pero para este análisis no se tendrá en cuenta).

1.4. Análisis de calidad final comparativo

Tras haber hecho todos los cambios en el código han quedado los siguientes resultados:



Se puede observar que a pesar de haber quitado fallos en “High” y en “Medium”, no se ha generado ningún fallo más aparte de los anteriores.
Es decir, se han arreglado 9 incidencias “High” y 5 incidencias “Medium”, pasando de 12 a 3 y de 23 a 18 respectivamente.
Otra cosa para destacar es que el Quality Gate antes de los errores da “Failed” mientras que al haber hecho todos los cambios me da “Passed”.